



UDF | Universidade do Distrito Federal

www.udf.edu.br



TRABALHO ACADÊMICO

Análise Comparativa de Algoritmos de Ordenação: Bubble Sort, Insertion Sort, Selection Sort e Quick Sort

Estudo de desempenho, particionamento e complexidade

INTEGRANTE

David Ribeiro de Sousa Alves

RGM: **42764971**

Atividade referente ao estudo dos métodos de ordenação Bubble, Insertion, Selection e Quick Sort.

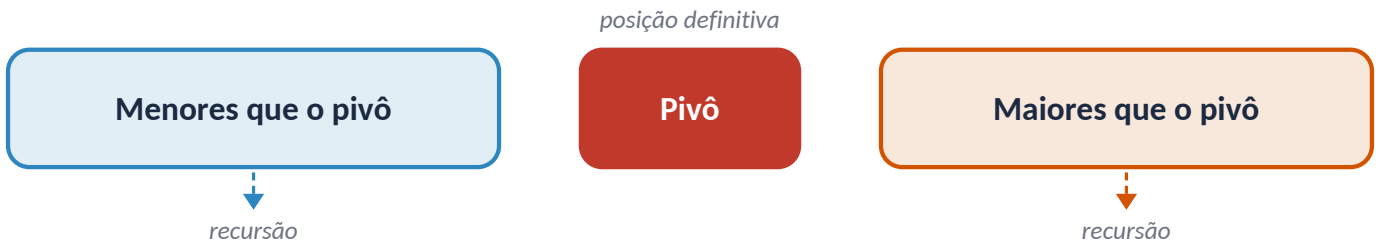
Brasília – DF, 2026

Quick Sort

Ordenação Rápida • estratégia Dividir para Conquistar

1 Ideia central

O **Quick Sort** (Ordenação Rápida) é um algoritmo altamente eficiente que se baseia na estratégia de “**Dividir para Conquistar**”. A premissa central é escolher um elemento do array chamado de “**pivô**” e usar esse pivô para dividir o array original em duas partes menores: uma contendo os elementos que são menores que o pivô, e outra contendo os elementos que são maiores. Esse particionamento resolve o problema aos poucos, ordenando recursivamente os sub-arrays resultantes.



2 Passo a passo do processo

1 Escolha do pivô

Seleciona-se um elemento do array para ser o pivô. Pode ser o primeiro, o último, o elemento do meio ou um escolhido aleatoriamente.

2 Particionamento

O array é reorganizado. Todos os elementos menores que o pivô são movidos para a sua esquerda, e todos os elementos maiores são movidos para a sua direita. Ao final dessa etapa, o pivô estará na sua posição definitiva no array ordenado.

3 Recursividade

O algoritmo é chamado recursivamente duas vezes: uma para o sub-array à esquerda do pivô e outra para o sub-array à direita.

4 Caso base

A recursão para quando os sub-arrays tiverem tamanho 0 ou 1, pois um array com um único elemento já está, por definição, ordenado.

3 Vantagens e desvantagens

Vantagens

- É um dos algoritmos de ordenação **mais rápidos na prática**.
- É um algoritmo **in-place**, ou seja, necessita de um espaço adicional mínimo (apenas o espaço da pilha de execução da recursividade), não exigindo a criação de novos arrays de cópia.
- Apresenta uma excelente **localidade de referência** para a memória cache do processador.

Desvantagens

- É um algoritmo **instável** (elementos com valores iguais podem ter sua ordem relativa original alterada durante o processo).
- Seu desempenho é **extremamente dependente da escolha do pivô**. Caso o menor ou o maior elemento seja sempre escolhido como pivô, sua eficiência cai drasticamente.
- Não é a melhor opção para listas que já estão quase ordenadas, a menos que um pivô aleatório ou o elemento central seja utilizado para evitar o pior caso.

4 Complexidade de tempo e espaço

TEMPO · MELHOR E MÉDIO CASO

$O(N \log N)$

Ocorre quando o pivô escolhido consegue dividir os elementos em duas metades perfeitamente ou quase perfeitamente equilibradas.

TEMPO · PIOR CASO

$O(N^2)$

Ocorre quando o array já está ordenado (ou em ordem reversa) e o pivô selecionado é sempre o último ou o primeiro elemento (criando partições desiguais de tamanho 0 e $N-1$).

ESPAÇO

$O(\log N)$

Para o caso médio, devido à profundidade da pilha de chamadas da recursão.

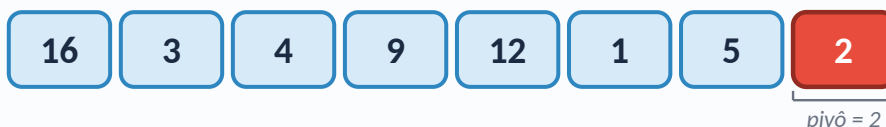
5 Exemplos práticos

Vamos utilizar a lista de números sugerida no documento base: [16, 3, 4, 9, 12, 1, 5, 2].

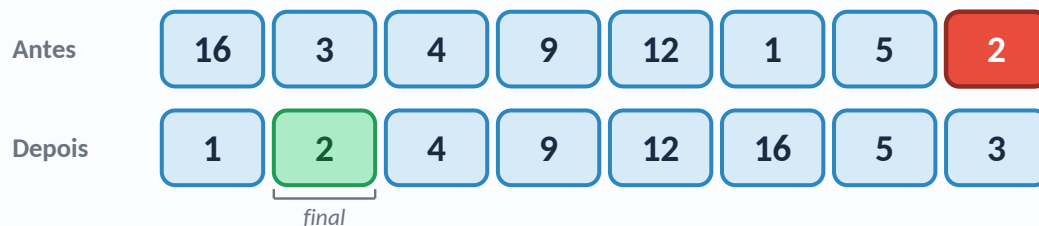
■ Pivô ■ Sub-array em processamento ■ Posição final (ordenado)

Passo 1 Escolher um pivô

Vetor



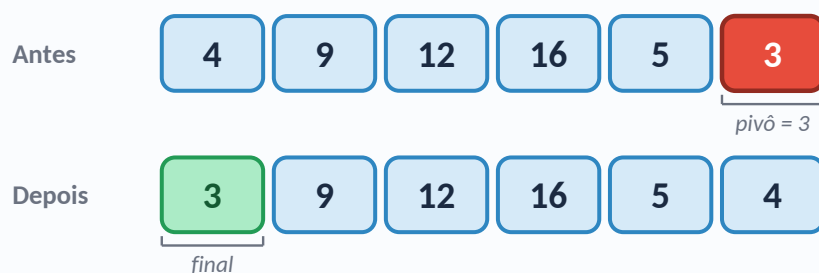
Escolhemos o último elemento como pivô: 2.

Passo 2 Particionar em relação ao 2

Apenas o 1 é menor que 2. O array se torna [1, 2, 4, 9, 12, 16, 5, 3]. O 2 encontrou sua posição final.

Passo 3 Recursão nos dois lados

Aplicamos a recursão do lado esquerdo [1] (já está ordenado) e do lado direito [4, 9, 12, 16, 5, 3].

Passo 4 Novo pivô no sub-array direito

No sub-array [4, 9, 12, 16, 5, 3], escolhemos o novo pivô: 3. Como não há números menores que 3 nesse sub-array, o 3 vai para o início dele: [3, 9, 12, 16, 5, 4].

Resultado

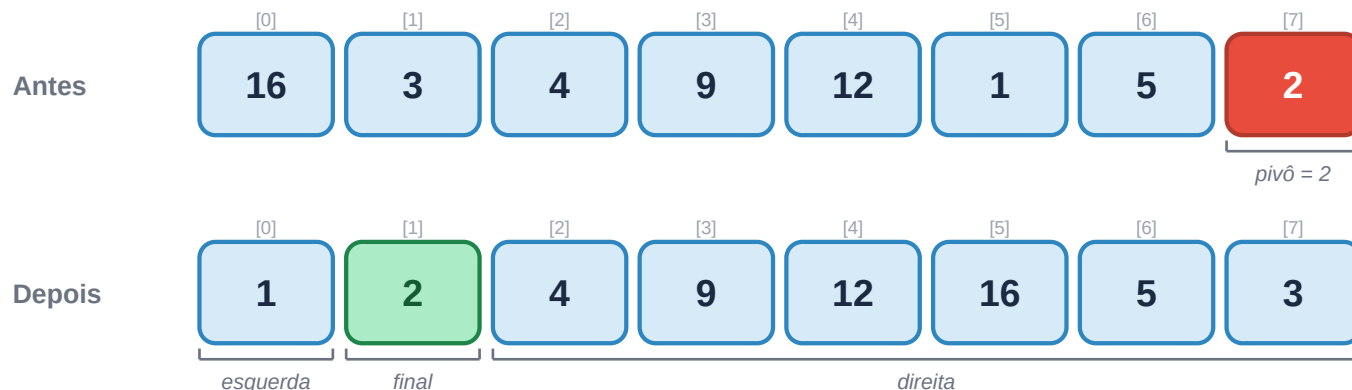
O processo continua recursivamente quebrando a lista, colocando os pivôs em seus devidos lugares, até resultar na lista ordenada: [1, 2, 3, 4, 5, 9, 12, 16].

Quick Sort - Passo a passo

Vetor: [16, 3, 4, 9, 12, 1, 5, 2] | pivô = último elemento

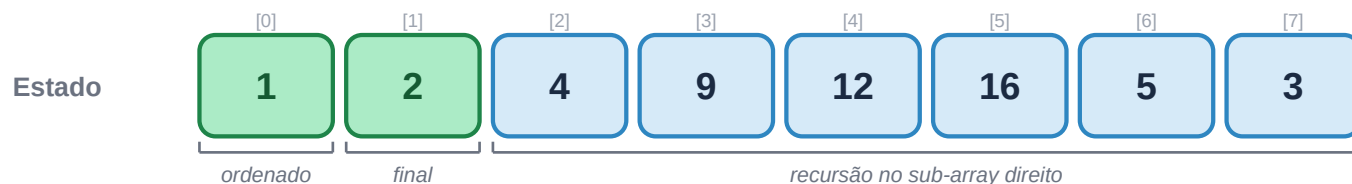
■ Pivô ■ Sub-array em processamento ■ Posição final (ordenado)

Passos 1 e 2 - Escolher o pivô e particionar



O pivô é o último elemento: **2**. Percorremos o vetor e movemos para a esquerda os valores menores que o pivô. Apenas o **1** é menor. Depois o pivô é colocado logo após ele: o **2** encontrou sua posição final.

Passo 3 - Recursão nos dois lados



Aplicamos a recursão em cada lado do 2. A esquerda **[1]** tem um só elemento, portanto já está ordenada. A direita **[4, 9, 12, 16, 5, 3]** segue para novo particionamento.

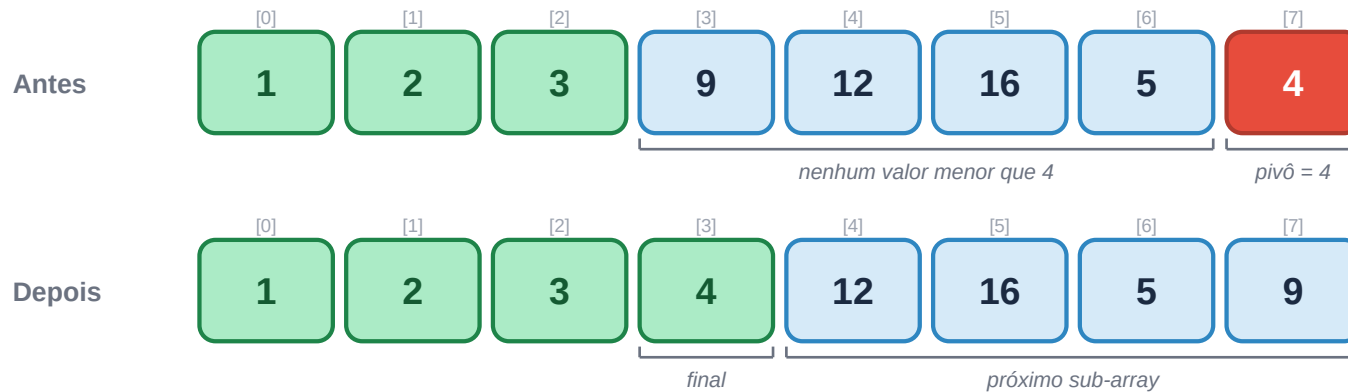
Passo 4 - Pivô 3 no sub-array direito



No sub-array **[4, 9, 12, 16, 5, 3]** o pivô é **3**. Nenhum número é menor que 3, então ele vai para o início do sub-array, trocando de lugar com o 4: **[3, 9, 12, 16, 5, 4]**.

■ Pivô ■ Sub-array em processamento ■ Posição final (ordenado)

Passo 5 - Pivô 4 em [9, 12, 16, 5, 4]



Novamente nenhum valor é menor que o pivô 4. Ele troca de lugar com o primeiro elemento do sub-array (o 9) e fixa sua posição: [4, 12, 16, 5, 9].

Passo 6 - Pivô 9 em [12, 16, 5, 9]



O pivô é 9. Apenas o 5 é menor que ele e vai para a esquerda; o 9 fica entre 5 e os maiores. Resultado do sub-array: [5, 9, 12, 16]. O 9 está na posição final.

■ Pivô ■ Sub-array em processamento ■ Posição final (ordenado)

Passo 7 - Pivô 16 em [12, 16]



O 5 sozinho já está ordenado. Em [12, 16] o pivô é 16: o 12 é menor e já está à esquerda, então nada muda. Sobra apenas [12], que é ordenado. A recursão termina.

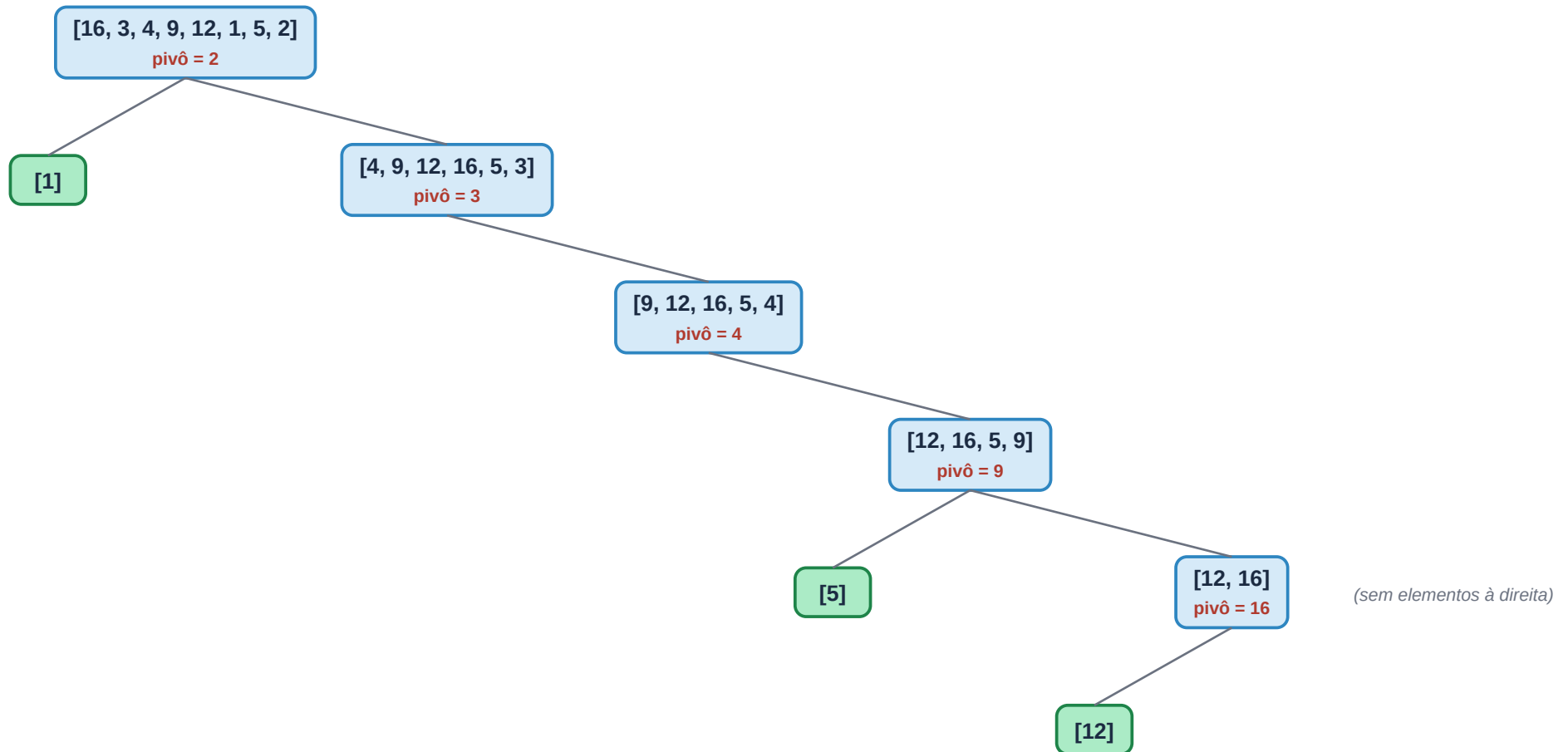
Resultado final

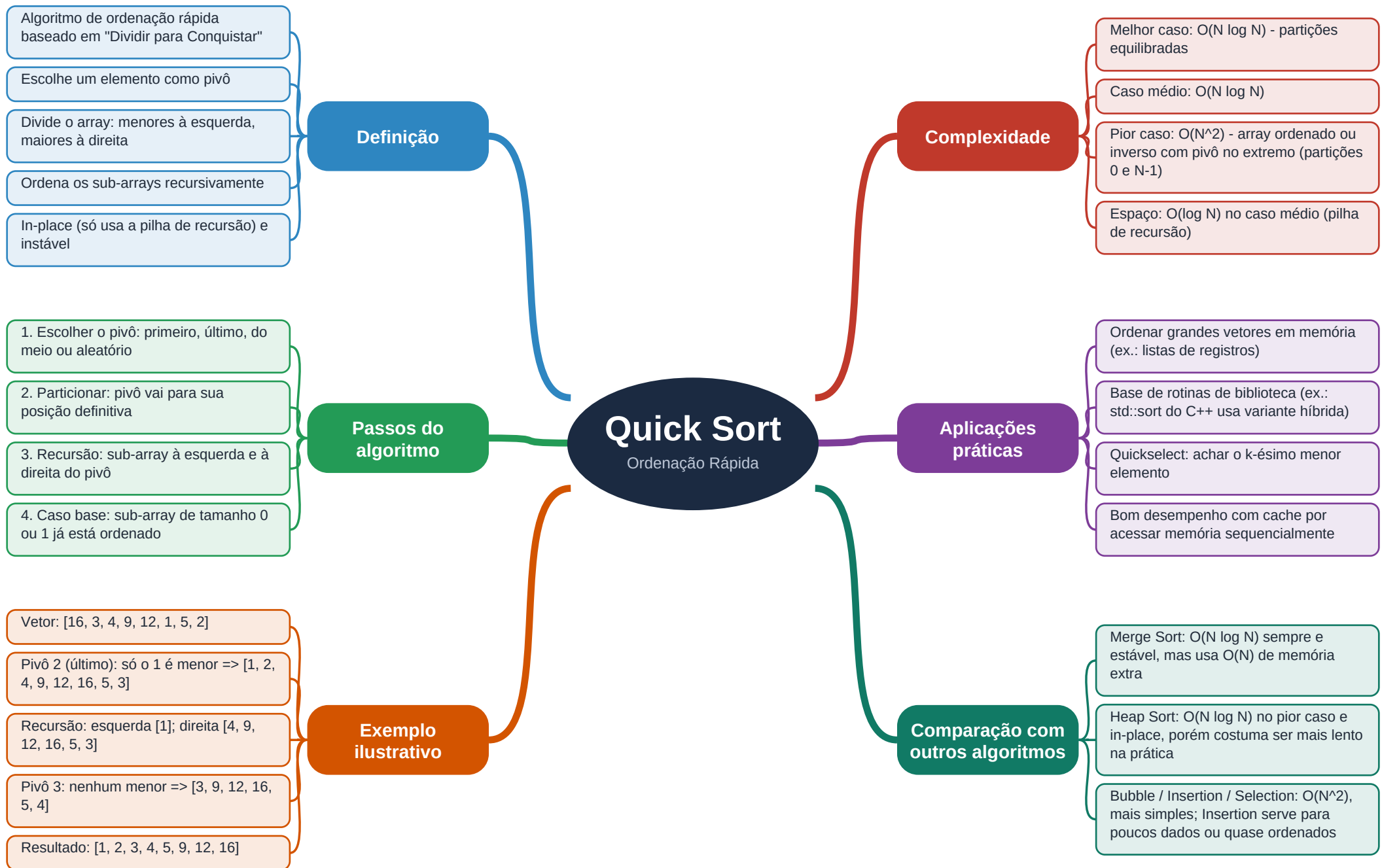


Cada partição fixa um pivô na posição definitiva e divide o problema em dois sub-arrays menores, até que todos tenham 0 ou 1 elemento.

Árvore de recursão

Cada nó mostra o sub-array; o pivô escolhido aparece em vermelho e vai para sua posição final.





SIMULAÇÃO DE ALGORITMOS DE ORDENAÇÃO

Bubble · Insertion · Selection · Quick Sort — com setas indicando trocas e deslocamentos

Lista utilizada (8 elementos)



Cada linha é uma fotografia do vetor após um passo do algoritmo. As **setas e ícones de troca** foram adicionados sobre cada linha para destacar exatamente quais posições foram comparadas, trocadas ou deslocadas naquele passo — facilitando acompanhar o "movimento" dos elementos de passo a passo. As cores indicam o papel de cada elemento.

- ☐ Ainda não ordenado
- ☒ Envolvido na troca deste passo
- ☐ Chave / elemento movido
- ☐ Pivô fixado
- ☐ Já na posição final

Página	Conteúdo
2	Bubble Sort — troca pares vizinhos; o maior valor sobe a cada passada ➦ setas entre vizinhos trocados
3	Insertion Sort — insere cada elemento na parte já ordenada ↪ seta da origem até o índice de inserção
4	Selection Sort — seleciona o menor e o coloca na posição certa ➦ seta dupla entre as posições trocadas
5	Quick Sort — escolhe um pivô e particiona ↔ chave do sub-array + seta no pivô fixado
6	Comparação — contagem de comparações e trocas de cada método

BUBBLE SORT

Trocas entre vizinhos, passada a passada — setas ↗ ↘ mostram cada troca

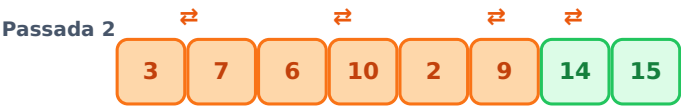
☐ Não trocado ☒ Trocado nesta passada ☒ Já na posição final



Lista original. A cada passada comparamos pares vizinhos e trocamos os que estão fora de ordem.



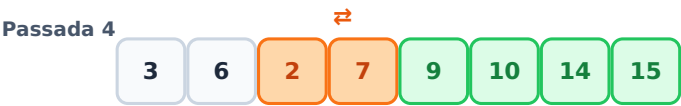
7 comparações, **6 trocas**: (14,7) (14,3) (14,10) (14,6) (15,2) (15,9). O maior valor do trecho (15) chega ao índice 7.



6 comparações, **4 trocas**: (7,3) (10,6) (14,2) (14,9). O maior valor do trecho (14) chega ao índice 6.



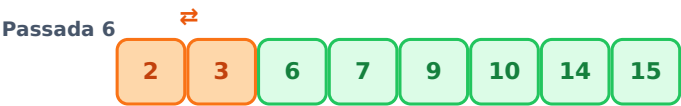
5 comparações, **3 trocas**: (7,6) (10,2) (10,9). O maior valor do trecho (10) chega ao índice 5.



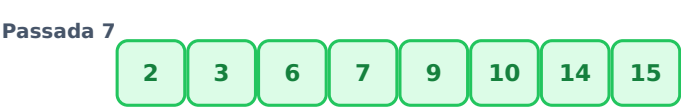
4 comparações, **1 troca**: (7,2). O maior valor do trecho (9) chega ao índice 4.



3 comparações, **1 troca**: (6,2). O maior valor do trecho (7) chega ao índice 3.



2 comparações, **1 troca**: (3,2). O maior valor do trecho (6) chega ao índice 2.



1 comparação e **nenhuma troca**. A lista está ordenada e o algoritmo para.

Após cada passada o maior valor restante fica no fim. Se uma passada não tem trocas, a lista já está ordenada.

INSERTION SORT

Inserção da chave na parte ordenada — seta mostra de onde a chave saiu e onde entrou

 Parte ordenada

 Chave inserida

 Ainda não processado



O primeiro elemento forma a parte ordenada (verde). Cada passo insere o próximo elemento (chave) no lugar certo dela.



Chave = 7: 1 comparação, 1 deslocamento para a direita.
Chave inserida no índice 0.



Chave = 3: 2 comparações, 2 deslocamentos para a direita.
Chave inserida no índice 0.



Chave = 10: 2 comparações, 1 deslocamento para a direita.
Chave inserida no índice 2.



Chave = 6: 4 comparações, 3 deslocamentos para a direita.
Chave inserida no índice 1.



Chave = 15: 1 comparação, já é maior que o anterior.
Fica no índice 5, sem deslocamentos.



Chave = 2: 6 comparações, 6 deslocamentos para a direita.
Chave inserida no índice 0.



Chave = 9: 4 comparações, 3 deslocamentos para a direita.
Chave inserida no índice 4.

A parte ordenada cresce um elemento por passo; elementos maiores que a chave são deslocados uma casa para a direita.

SELECTION SORT

Seleção do menor elemento a cada passo — seta dupla ⇄
liga as duas posições trocadas

☒ Já na posição final

☐ Menor encontrado / posição a preencher

☐ Ainda não ordenado



Procura o menor em [0..7] (7 comparações): **2** no índice 6. Troca com o **14** do índice 0.



Procura o menor em [1..7] (6 comparações): **3** no índice 2. Troca com o **7** do índice 1.



Procura o menor em [2..7] (5 comparações): **6** no índice 4. Troca com o **7** do índice 2.



Procura o menor em [3..7] (4 comparações): **7** no índice 4. Troca com o **10** do índice 3.



Procura o menor em [4..7] (3 comparações): **9** no índice 7. Troca com o **10** do índice 4.



Procura o menor em [5..7] (2 comparações): **10** no índice 7. Troca com o **15** do índice 5.



Procura o menor em [6..7] (1 comparação): **14** já está no índice 6. Sem troca. Lista ordenada.

Se a posição i e o menor coincidem, não há troca. Este método faz no máximo $N-1$ trocas.

QUICK SORT

Pivô = último elemento; chave $\Leftarrow$ marca o sub-array e a seta ∇ mostra onde o pivô foi fixado

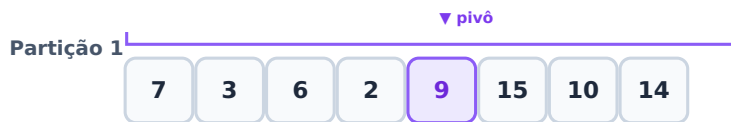
☒ Já na posição final

☐ Pivô fixado agora

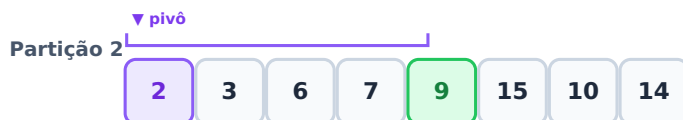
☐ Sub-array a ordenar



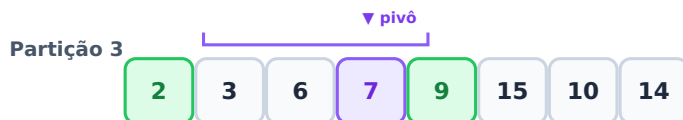
Vetor inicial. Pivô = último elemento = 9.



Partição de [14,7,3,10,6,15,2,9], pivô 9. Menores ou iguais: 7,3,6,2.
Pivô fixado no índice 4. Próximo pivô: 2.



Partição de [7,3,6,2], pivô 2. Menores ou iguais: nenhum.
Pivô fixado no índice 0. Próximo pivô: 7.



Partição de [3,6,7], pivô 7. Menores ou iguais: 3,6.
Pivô fixado no índice 3. Próximo pivô: 6.



Partição de [3,6], pivô 6. Menores ou iguais: 3.
Pivô fixado no índice 2. Próximo pivô: 14.



Partição de [15,10,14], pivô 14. Menores ou iguais: 10.
Pivô fixado no índice 6. Todos os sub-arrays restantes têm 1 elemento — lista ordenada.

Cada linha mostra o vetor logo após uma partição. Sub-arrays com um só elemento já contam como ordenados.

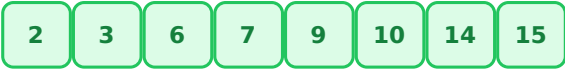
COMPARAÇÃO DOS MÉTODOS

Mesma lista: [14, 7, 3, 10, 6, 15, 2, 9]

Quantidade de operações na simulação

Algoritmo	Comparações	Trocas / deslocamentos	Observação
Bubble Sort	28	16 trocas	Muitas trocas entre vizinhos.
Insertion Sort	20	16 deslocamentos	Bom em listas quase ordenadas.
Selection Sort	28	6 trocas	Poucas trocas (no máximo N−1).
Quick Sort	15	8 trocas	Divide em sub-arrays menores.

Resultado final (igual nos quatro métodos)



Contagens feitas para esta lista específica e com a versão de cada algoritmo descrita nas páginas anteriores; outras listas dão outros valores.

3

CÓDIGO-FONTE

Atividade 3

Atividade 3.cpp

```

1  #include <stdio.h>
2  #include <vector>
3  #include <cstdlib>
4  #include <ctime>
5  #include <cstdio>
6  using namespace std;
7
8  // Armazena as contagens de operações
9  struct ContadorOperacoes {
10     long long totalComparacoes = 0;
11     long long totalTrocas = 0;
12 };
13 // Algoritmo Bubble Sort
14 void executarBubbleSort(vector<int>& vetor, ContadorOperacoes& contador) {
15     int tamanhoVetor = vetor.size();
16     for (int passada = 0; passada < tamanhoVetor - 1; passada++) {
17         bool houveTroca = false;
18         for (int indiceAtual = 0; indiceAtual < tamanhoVetor - passada - 1; indiceAtual++) {
19             contador.totalComparacoes++;
20             if (vetor[indiceAtual] > vetor[indiceAtual + 1]) {
21                 swap(vetor[indiceAtual], vetor[indiceAtual + 1]);
22                 contador.totalTrocas++;
23                 houveTroca = true;
24             }
25         }
26         // Interrompe se o vetor já estiver ordenado
27         if (!houveTroca)
28             break;
29     }
30 }
31 // Função de particionamento do Quick Sort
32 int particionarVetor(vector<int>& vetor, int indiceInicio, int indiceFim,
33                     ContadorOperacoes& contador) {
34     int valorPivo = vetor[indiceFim];
35     int indiceMenores = indiceInicio - 1;
36     for (int indiceAtual = indiceInicio; indiceAtual < indiceFim; indiceAtual++) {
37         contador.totalComparacoes++;
38         if (vetor[indiceAtual] <= valorPivo) {
39             indiceMenores++;
40             swap(vetor[indiceMenores], vetor[indiceAtual]);
41             contador.totalTrocas++;
42         }
43     }
44     // Posiciona o pivô no local correto
45     swap(vetor[indiceMenores + 1], vetor[indiceFim]);
46     contador.totalTrocas++;
47     return indiceMenores + 1;
48 }
49 // Algoritmo Quick Sort
50 void executarQuickSort(vector<int>& vetor, int indiceInicio, int indiceFim,
51                       ContadorOperacoes& contador) {
52     if (indiceInicio < indiceFim) {
53         int posicaoPivo = particionarVetor(vetor, indiceInicio, indiceFim, contador);
54         // Ordena as sub-listas da esquerda e direita
55         executarQuickSort(vetor, indiceInicio, posicaoPivo - 1, contador);
56         executarQuickSort(vetor, posicaoPivo + 1, indiceFim, contador);
57     }
58 }
59

```

```

60 // Gera números aleatórios entre 0 e 9999
61 vector<int> gerarVetorAleatorio(int tamanho) {
62     vector<int> vetorGerado(tamanho);
63
64     for (int i = 0; i < tamanho; i++) {
65         vetorGerado[i] = rand() % 10000;
66     }
67     return vetorGerado;
68 }
69 int main() {
70     // Semente para geração aleatória
71     srand(time(0));
72     vector<int> tamanhosTeste = {10, 20, 1000};
73     // Cabeçalho da tabela
74     printf("%-18s %-25s %-25s %-25s %-25s\n",
75         "Tamanho do Array", "Bubble Sort - Comp.", "Bubble Sort - Trocas", "Quick Sort - Comp.",
76         "Quick Sort - Mov.");
77
78     for (int tamanho : tamanhosTeste) {
79         // Gera vetor original
80         vector<int> vetorBase = gerarVetorAleatorio(tamanho);
81         // Cria cópias idênticas
82         vector<int> vetorParaBubble = vetorBase;
83         vector<int> vetorParaQuick = vetorBase;
84         ContadorOperacoes contadorBubble;
85         ContadorOperacoes contadorQuick;
86         // Executa Bubble Sort
87         executarBubbleSort(vetorParaBubble, contadorBubble);
88
89         // Executa Quick Sort
90         executarQuickSort(vetorParaQuick, 0, vetorParaQuick.size() - 1, contadorQuick );
91         // Exibe os resultados
92         printf("%-18d %-25lld %-25lld %-25lld %-25lld\n",
93             tamanho,
94             contadorBubble.totalComparacoes,
95             contadorBubble.totalTrocas,
96             contadorQuick.totalComparacoes,
97             contadorQuick.totalTrocas);
98     }
99     return 0;
100 }

```

3

EXECUÇÃO

Atividade 3

[print original](#)

```

1 #include <stdio.h>
2 #include <vector>
3 #include <cstdlib>
4 #include <ctime>
5 #include <cstdlib>
6 using namespace std;
7
8 // Armazena as contagens de operações
9 struct ContadorOperacoes {
10     long long totalComparacoes = 0;
11     long long totalTrocas = 0;
12 };
13 // Algoritmo Bubble Sort
14 void executarBubbleSort(vector<int>& vetor, ContadorOperacoes& contador) {
15     int tamanhoVetor = vetor.size();
16     for (int passada = 0; passada < tamanhoVetor - 1; passada++) {
17         bool houveTroca = false;
18         for (int indiceAtual = 0; indiceAtual < tamanhoVetor - passada - 1; indiceAtual++) {
19             contador.totalComparacoes++;
20             if (vetor[indiceAtual] > vetor[indiceAtual + 1]) {
21                 swap(vetor[indiceAtual], vetor[indiceAtual + 1]);
22                 contador.totalTrocas++;
23                 houveTroca = true;
24             }
25         }
26         // interrompe se o vetor já estiver ordenado
27         if (!houveTroca) break;
28     }
29 }
30
31 // Função de particionamento do Quick Sort
32 int particionarVetor(vector<int>& vetor, int indiceInicio, int indiceFim,
33                     ContadorOperacoes& contador) {
34     int valorPivo = vetor[indiceFim];
35     int indiceMenores = indiceInicio - 1;
36     for (int indiceAtual = indiceInicio; indiceAtual < indiceFim; indiceAtual++) {
37         contador.totalComparacoes++;
38         if (vetor[indiceAtual] <= valorPivo) {
39             indiceMenores++;
40             swap(vetor[indiceAtual], vetor[indiceMenores]);
41         }
42     }
43     swap(vetor[indiceMenores + 1], vetor[indiceFim]);
44     return indiceMenores + 1;
45 }
46
47 int main() {
48     int tamanhoArray;
49     printf("Digite o tamanho do array: ");
50     scanf("%d", &tamanhoArray);
51
52     vector<int> vetor(tamanhoArray);
53     for (int i = 0; i < tamanhoArray; i++) {
54         vetor[i] = rand() % 1000;
55     }
56
57     ContadorOperacoes contador;
58     executarBubbleSort(vetor, contador);
59     printf("Bubble Sort - Comp.: %lld\n", contador.totalComparacoes);
60     printf("Bubble Sort - Trocas: %lld\n", contador.totalTrocas);
61
62     contador = ContadorOperacoes();
63     particionarVetor(vetor, 0, tamanhoArray - 1, contador);
64     printf("Quick Sort - Comp.: %lld\n", contador.totalComparacoes);
65     printf("Quick Sort - Mov.: %lld\n", contador.totalTrocas);
66
67     return 0;
68 }

```

Tamanho do Array	Bubble Sort - Comp.	Bubble Sort - Trocas	Quick Sort - Comp.	Quick Sort - Mov.
10	39	15	25	22
20	180	110	64	48
1000	499380	237222	10758	6440

...Program finished with exit code 0
Press ENTER to exit console.

4

CÓDIGO-FONTE

Atividade 4.0

Atividade 4.0.cpp

```

1  #include <stdio.h>
2  #include <iostream>
3  #include <vector>
4  #include <string>
5  #include <algorithm>
6
7  using namespace std;
8  // Estrutura para rastrear o desempenho
9  struct ContadorOperacoes {
10     long long totalComparacoes = 0;
11     long long totalTrocas = 0;
12 };
13
14 // 1. Bubble Sort para Strings
15 void executarBubbleSort(vector<string>& vetor, ContadorOperacoes& contador) {
16     int tamanho = vetor.size();
17     for (int passada = 0; passada < tamanho - 1; passada++) {
18         bool houveTroca = false;
19         for (int i = 0; i < tamanho - passada - 1; i++) {
20             contador.totalComparacoes++;
21             // Comparação alfabética direta utilizando '>'
22             if (vetor[i] > vetor[i + 1]) {
23                 swap(vetor[i], vetor[i + 1]);
24                 contador.totalTrocas++;
25                 houveTroca = true;
26             }
27         }
28         if (!houveTroca) break;
29     }
30 }
31
32 // 2. Insertion Sort para Strings
33 void executarInsertionSort(vector<string>& vetor, ContadorOperacoes& contador) {
34     int n = vetor.size();
35     for (int i = 1; i < n; i++) {
36         string chave = vetor[i];
37         int j = i - 1;
38         while (j >= 0) {
39             contador.totalComparacoes++;
40             if (vetor[j] > chave) {
41                 vetor[j + 1] = vetor[j];
42                 contador.totalTrocas++;
43                 j--;
44             } else {
45                 break;
46             }
47         }
48         vetor[j + 1] = chave;
49     }
50 }
51
52 // 3. Selection Sort para Strings
53 void executarSelectionSort(vector<string>& vetor, ContadorOperacoes& contador) {
54     int n = vetor.size();
55     for (int i = 0; i < n - 1; i++) {
56         int min_idx = i;
57         for (int j = i + 1; j < n; j++) {
58             contador.totalComparacoes++;
59             if (vetor[j] < vetor[min_idx]) {

```

```

60         min_idx = j;
61     }
62 }
63 if (min_idx != i) {
64     swap(vetor[i], vetor[min_idx]);
65     contador.totalTrocas++;
66 }
67 }
68 }
69
70 // Particionamento do Quick Sort para Strings
71 int particionarVetor(vector<string>& vetor, int indiceInicio, int indiceFim, ContadorOperacoes&
    contador) {
72     string valorPivo = vetor[indiceFim];
73     int indiceMenores = indiceInicio - 1;
74     for (int indiceAtual = indiceInicio; indiceAtual < indiceFim; indiceAtual++) {
75         contador.totalComparacoes++;
76         if (vetor[indiceAtual] <= valorPivo) {
77             indiceMenores++;
78             swap(vetor[indiceMenores], vetor[indiceAtual]);
79             contador.totalTrocas++;
80         }
81     }
82     swap(vetor[indiceMenores + 1], vetor[indiceFim]);
83     contador.totalTrocas++;
84     return indiceMenores + 1;
85 }
86
87 // 4. Quick Sort para Strings
88 void executarQuickSort(vector<string>& vetor, int indiceInicio, int indiceFim, ContadorOperacoes
    & contador) {
89     if (indiceInicio < indiceFim) {
90         int posicaoPivo = particionarVetor(vetor, indiceInicio, indiceFim, contador);
91         executarQuickSort(vetor, indiceInicio, posicaoPivo - 1, contador);
92         executarQuickSort(vetor, posicaoPivo + 1, indiceFim, contador);
93     }
94 }
95
96 int main() {
97     // Lista com 50 palavras (Nomes de Marcas com inicial maiúscula para manter precisão na
    tabela ASCII)
98     vector<string> listaMarcas = {
99         "Apple", "Samsung", "Nike", "Adidas", "Microsoft", "Google", "Amazon", "Toyota", "Honda"
    , "Ford",
100     "Chevrolet", "Coca-cola", "Pepsi", "Mcdonalds", "Burgerking", "Sony", "Nintendo", "Plays
    tation", "Xbox", "Asus",
101     "Acer", "Lenovo", "Hp", "Dell", "Intel", "Amd", "Nvidia", "Logitech", "Corsair", "Razer"
    ,
102     "Ferrari", "Porsche", "Bmw", "Audi", "Mercedes", "Volkswagen", "Fiat", "Renault", "Peuge
    ot", "Citroen",
103     "Zara", "Gucci", "Prada", "Rolex", "Casio", "Seiko", "Yamaha", "Kawasaki", "Suzuki", "Ha
    rley-davidson"
104 };
105
106 // Criação de cópias independentes
107 vector<string> vetorBubble = listaMarcas;
108 vector<string> vetorInsertion = listaMarcas;
109 vector<string> vetorSelection = listaMarcas;
110 vector<string> vetorQuick = listaMarcas;
111
112 ContadorOperacoes contBubble, contInsertion, contSelection, contQuick;
113
114 // Aplicação dos algoritmos
115 executarBubbleSort(vetorBubble, contBubble);
116 executarInsertionSort(vetorInsertion, contInsertion);

```

```

117     executarSelectionSort(vetorSelection, contSelection);
118     executarQuickSort(vetorQuick, 0, vetorQuick.size() - 1, contQuick);
119
120     // Impressão da Tabela de Desempenho
121     printf("\nDesempenho na Ordenacao de 50 Palavras:\n\n");
122     printf("%20s %15s %15s\n", "Algoritmo", "Comparacoes", "Trocas/Mov.");
123     printf("-----\n");
124     printf("%20s %15lld %15lld\n", "Bubble Sort", contBubble.totalComparacoes, contBubble.totalT
        rocas);
125     printf("%20s %15lld %15lld\n", "Insertion Sort", contInsertion.totalComparacoes, contInserti
        on.totalTrocas);
126     printf("%20s %15lld %15lld\n", "Selection Sort", contSelection.totalComparacoes, contSelecti
        on.totalTrocas);
127     printf("%20s %15lld %15lld\n", "Quick Sort", contQuick.totalComparacoes, contQuick.totalTroc
        as);
128     printf("-----\n\n");
129
130     // Demonstração visual de que a lista foi ordenada alfabeticamente (mostrando os 5
        primeiros)
131     printf("Visualizacao elementps:\n");
132     for(int i = 0; i < 50; i++) {
133         cout << i+1 << ". " << vetorQuick[i] << endl;
134     }
135
136     return 0;
137 }

```

4

EXECUÇÃO

Atividade 4

print recortado

```
main.cpp
```

```
Desempenho na Ordenacao de 50 Palavras:
```

Algoritmo	Comparacoes	Trocas/Mov.
Bubble Sort	1134	500
Insertion Sort	547	500
Selection Sort	1225	44
Quick Sort	219	119

```
Visualizacao elementps:
1. Acer
2. Adidas
3. Amazon
4. Amd
5. Apple
6. Asus
7. Audi
8. Bmw
9. Burgerking
10. Casio
11. Chevrolet
12. Citroen
13. Coca-cola
14. Corsair
15. Dell
16. Ferrari
17. Fiat
18. Ford
19. Google
20. Gucci
21. Harley-davidson
22. Honda
23. Hp
24. Intel
25. Kawasaki
26. Lenovo
27. Logitech
28. Mcdonalds
29. Mercedes
30. Microsoft
31. Nike
32. Nintendo
33. Nvidia
34. Pepsi
35. Peugeot
36. Playstation
37. Porsche
38. Prada
39. Razer
40. Renault
41. Rolex
42. Samsung
43. Seiko
44. Sony
45. Suzuki
46. Toyota
47. Volkswagen
48. Xbox
49. Yamaha
50. Zara

...Program finished with exit code 0
Press ENTER to exit console.
```

5

CÓDIGO-FONTE

Atividade 5

Atividade 5.cpp

```

1  #include <stdio.h>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  // Armazena as contagens de operações
8  struct ContadorOperacoes {
9      long long totalComparacoes = 0;
10     long long totalTrocas = 0;
11 };
12
13 // 1. Algoritmo Bubble Sort (com otimização de paragem)
14 void executarBubbleSort(vector<int>& vetor, ContadorOperacoes& contador) {
15     int tamanhoVetor = vetor.size();
16     for (int passada = 0; passada < tamanhoVetor - 1; passada++) {
17         bool houveTroca = false;
18         for (int indiceAtual = 0; indiceAtual < tamanhoVetor - passada - 1; indiceAtual++) {
19             contador.totalComparacoes++;
20             if (vetor[indiceAtual] > vetor[indiceAtual + 1]) {
21                 swap(vetor[indiceAtual], vetor[indiceAtual + 1]);
22                 contador.totalTrocas++;
23                 houveTroca = true;
24             }
25         }
26         // Interrompe se o vetor já estiver ordenado
27         if (!houveTroca)
28             break;
29     }
30 }
31
32 // 2. Algoritmo Insertion Sort
33 void executarInsertionSort(vector<int>& vetor, ContadorOperacoes& contador) {
34     int n = vetor.size();
35     for (int i = 1; i < n; i++) {
36         int chave = vetor[i];
37         int j = i - 1;
38         while (j >= 0) {
39             contador.totalComparacoes++;
40             if (vetor[j] > chave) {
41                 vetor[j + 1] = vetor[j];
42                 contador.totalTrocas++; // Contabilizado como deslocamento/troca
43                 j--;
44             } else {
45                 break;
46             }
47         }
48         vetor[j + 1] = chave;
49     }
50 }
51
52 // 3. Algoritmo Selection Sort
53 void executarSelectionSort(vector<int>& vetor, ContadorOperacoes& contador) {
54     int n = vetor.size();
55     for (int i = 0; i < n - 1; i++) {
56         int min_idx = i;
57         for (int j = i + 1; j < n; j++) {
58             contador.totalComparacoes++;
59             if (vetor[j] < vetor[min_idx]) {

```

```

60         min_idx = j;
61     }
62 }
63 if (min_idx != i) {
64     swap(vetor[i], vetor[min_idx]);
65     contador.totalTrocas++;
66 }
67 }
68 }
69
70 // Função de particionamento do Quick Sort
71 int particionarVetor(vector<int>& vetor, int indiceInicio, int indiceFim, ContadorOperacoes& contador) {
72     int valorPivo = vetor[indiceFim];
73     int indiceMenores = indiceInicio - 1;
74     for (int indiceAtual = indiceInicio; indiceAtual < indiceFim; indiceAtual++) {
75         contador.totalComparacoes++;
76         if (vetor[indiceAtual] <= valorPivo) {
77             indiceMenores++;
78             swap(vetor[indiceMenores], vetor[indiceAtual]);
79             contador.totalTrocas++;
80         }
81     }
82     swap(vetor[indiceMenores + 1], vetor[indiceFim]);
83     contador.totalTrocas++;
84     return indiceMenores + 1;
85 }
86
87 // 4. Algoritmo Quick Sort
88 void executarQuickSort(vector<int>& vetor, int indiceInicio, int indiceFim, ContadorOperacoes& contador) {
89     if (indiceInicio < indiceFim) {
90         int posicaoPivo = particionarVetor(vetor, indiceInicio, indiceFim, contador);
91         executarQuickSort(vetor, indiceInicio, posicaoPivo - 1, contador);
92         executarQuickSort(vetor, posicaoPivo + 1, indiceFim, contador);
93     }
94 }
95
96 int main() {
97     // Definição da lista quase ordenada
98     vector<int> listaBase = {1, 2, 3, 8, 5, 6, 7, 4};
99
100
101     // Criação de cópias independentes da lista original.
102     // Isso é necessário porque os algoritmos alteram (ordenam) a lista original.
103     // Sem essas cópias, o segundo algoritmo receberia a lista já ordenada pelo primeiro,
104     // o que invalidaria o teste de desempenho para o cenário quase ordenado.
105     vector<int> vetorBubble = listaBase;
106     vector<int> vetorInsertion = listaBase;
107     vector<int> vetorSelection = listaBase;
108     vector<int> vetorQuick = listaBase;
109
110     ContadorOperacoes contBubble, contInsertion, contSelection, contQuick;
111
112     // Execução
113     executarBubbleSort(vetorBubble, contBubble);
114     executarInsertionSort(vetorInsertion, contInsertion);
115     executarSelectionSort(vetorSelection, contSelection);
116     executarQuickSort(vetorQuick, 0, vetorQuick.size() - 1, contQuick);
117
118     // Impressão dos resultados
119     // Formatação para criar uma visualização em formato de tabela:
120     printf("Resultados para a lista parcialmente ordenada [1, 2, 3, 8, 5, 6, 7, 4]:\n\n");
121
122     printf("%20s %15s %15s\n", "Algoritmo", "Comparacoes", "Trocas/Mov.");

```

```
123     printf("-----\n");
124     printf("%20s %15lld\n", "Insertion Sort", contInsertion.totalComparacoes, contInsertion.totalTrocas);
125     printf("%20s %15lld %15lld\n", "Bubble Sort", contBubble.totalComparacoes, contBubble.totalTrocas);
126     printf("%20s %15lld %15lld\n", "Selection Sort", contSelection.totalComparacoes, contSelection.totalTrocas);
127     printf("%20s %15lld %15lld\n", "Quick Sort", contQuick.totalComparacoes, contQuick.totalTrocas);
128     printf("-----\n");
129
130     return 0;
131 }
```

5

EXECUÇÃO

Atividade 5

[print original](#)

```

1 #include <stdio.h>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 // Armazena as contagens de operações
8 struct ContadorOperacoes {
9     long long totalComparacoes = 0;
10    long long totalTrocas = 0;
11 };
12
13 // 1. Algoritmo Bubble Sort (com otimização de paragem)
14 void executarBubbleSort(vector<int>& vetor, ContadorOperacoes& contador) {
15     int tamanhoVetor = vetor.size();
16     for (int passada = 0; passada < tamanhoVetor - 1; passada++) {
17         bool houveTroca = false;
18         for (int indiceAtual = 0; indiceAtual < tamanhoVetor - passada - 1; indiceAtual++) {
19             contador.totalComparacoes++;
20             if (vetor[indiceAtual] > vetor[indiceAtual + 1]) {
21                 swap(vetor[indiceAtual], vetor[indiceAtual + 1]);
22                 contador.totalTrocas++;
23                 houveTroca = true;
24             }
25         }
26         // Interrompe se o vetor já estiver ordenado
27         if (!houveTroca)
28             break;
29     }
30 }
31
32 // 2. Algoritmo Insertion Sort

```

input

Resultados para a lista parcialmente ordenada [1, 2, 3, 8, 5, 6, 7, 4]:

Algoritmo	Comparacoes	Trocas/Mov.
Insertion Sort	14	
Bubble Sort	25	7
Selection Sort	28	1
Quick Sort	16	18

...Program finished with exit code 0
Press ENTER to exit console.